

Practical 1: Define a Simple Service Link Converting INR to Dollar and Call It from Different Platforms (Java C .NET)

Objective: To build a simple service that converts INR to USD, deploy it on the cloud, and then access it using both Java and .NET platforms.

Step-by-Step Process

1. Define the Service (API) for INR to USD Conversion

We'll start by building a RESTful API for currency conversion. For simplicity, this will return a hardcoded conversion rate. You could later integrate it with an external API to fetch real-time rates.

Using Python Flask (for simplicity):

```
# app.py
```

```
from flask import Flask, jsonify, request
```

```
app = Flask(__name__)
```

```
@app.route('/convert', methods=['GET']) def
convert_inr_to_usd():
    # Hardcoded conversion rate for INR to USD conversion_rate =
    0.012 # Example: 1 INR = 0.012 USD try:
    # Get the amount in INR from the query parameter
    inr_amount = float(request.args.get('inr'))
    usd_amount = inr_amount * conversion_rate
    return jsonify({
        'inr': inr_amount, 'usd':
        usd_amount,
        'conversion_rate': conversion_rat
    })

except ValueError:

    return jsonify({'error': 'Invalid INR amount'}), 400
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

Deploy the API to Cloud (e.g., AWS, Azure, or Heroku)

For simplicity, we'll deploy this service to **Heroku**.

- Install the necessary tools for Heroku:

- Install Heroku CLI
- Deploy to Heroku:

```
# Create a requirements.txt file for Heroku echo  
"Flask" > requirements.txt
```

```
# Initialize Git (if not initialized) git init  
git add .  
  
git commit -m "Initial commit"
```

```
# Login to Heroku heroku login
```

```
# Create a Heroku app  
  
heroku create currency-conversion-api
```

```
# Push your code to Heroku
```

```
git push heroku master
```

```
# Open the app in the browser heroku  
open
```

This will deploy your API to a public URL on Heroku, such as <https://currency-conversion-api.herokuapp.com/>.

Call the API from Java

Now, let's create a Java client to call this API.

1. Java Client Code (using HttpURLConnection)

```
import java.io.InputStreamReader; import  
java.io.BufferedReader; import  
java.net.HttpURLConnection; import  
java.net.URL;
```

```
public class CurrencyConverter { public static void  
    main(String[] args) {  
        try {
```

```

String inrAmount = "100"; // Example INR amount

String url = "https://currency-conversion-api.herokuapp.com/convert?inr=" +
inrAmount;

// Create a connection to the API

URLConnection connection = (URLConnection) new URL(url).openConnection();
connection.setRequestMethod("GET");
// Read the response
BufferedReader in = new BufferedReader(new
InputStreamReader(connection.getInputStream()));

String inputLine;

StringBuffer response = new StringBuffer();

while ((inputLine = in.readLine()) != null)
{
    response.append(inputLine);
}

in.close();
// Print the response

System.out.println("Response: " + response.toString());

} catch (Exception e) { e.printStackTrace();
}

}

}

```

Steps:

- Replace <https://currency-conversion-api.herokuapp.com/convert?inr=100> with the URL of your deployed service.
- When you run this Java program, it should print the result of the INR to USD conversion.

Call the API from .NET

For .NET, we can use HttpClient to call the API.

1. C# Client Code (using HttpClient)

csharp Copy

```
using System;

using System.Net.Http;

using System.Threading.Tasks;

class CurrencyConverter

{

    static async Task Main(string[] args)

    {

        string inrAmount = "100"; // Example INR amount

        string url = "https://currency-conversion-api.herokuapp.com/convert?inr=" +

inrAmount;

        using (HttpClient client = new HttpClient())

        {

            try

            {

                // Make the API call

                HttpResponseMessage response = await client.GetAsync(url);

                // Ensure successful response

                response.EnsureSuccessStatusCode();

                // Read the response content as string

                string responseBody = await response.Content.ReadAsStringAsync();

                // Output the response

                Console.WriteLine("Response: " + responseBody);

            }

        }

    }

}
```

```
catch (Exception e)
{
    Console.WriteLine("Error: " + e.Message);
}
}
}
}
```

Steps:

- Replace <https://currency-conversion-api.herokuapp.com/convert?inr=100> with the URL of your deployed service.
- Running this program will print the result of the INR to USD conversion in the console.

Summary of the Process

1. **Create a simple API** for INR to USD conversion using Python and Flask.
2. **Deploy the API** to a cloud platform (like Heroku).
3. **Build client applications** in Java and .NET that call the API using HTTP.
4. **Test the clients** to ensure they can fetch the conversion result from the service.

Next Steps

- You can extend the API to integrate with a real-time currency conversion service.
- Secure the API with authentication (e.g., OAuth, API keys).

Practical 2: Create a Simple SOAP Service

SOAP (Simple Object Access Protocol) is a protocol for exchanging structured information in the implementation of web services. SOAP is highly extensible and allows

for operations to be called over a network. Here, we will create a simple SOAP service that converts INR (Indian Rupee) to USD (Dollar).

We will break down the process into the following steps:

Steps to Create a SOAP Service

1. Set Up the Environment

To create a SOAP web service, we can use **Java** along with **Apache CXF** or **JAX-WS** (Java API for XML Web Services).

For this, we'll use **Apache CXF** as it provides a simple and fast way to create SOAP web services.

- **Install Apache CXF** (If you're using Maven, you don't need to install Apache CXF directly, just add dependencies).

```
<dependencies>
  <dependency>
    <groupId>org.apache.cxf</groupId>
    <artifactId>cxf-rt-frontend-jaxws</artifactId>
    <version>3.4.5</version>
  </dependency>
  <dependency>
    <groupId>org.apache.cxf</groupId>
    <artifactId>cxf-rt-transport-http</artifactId>
    <version>3.4.5</version>
  </dependency>
  <dependency>
    <groupId>javax.xml.ws</groupId>
    <artifactId>jaxws-api</artifactId>
    <version>2.3.0</version>
  </dependency>
</dependencies>
```

Create the SOAP Web Service

Step 2.1: Define the Service Endpoint Interface (SEI)

The SEI will define the methods that will be exposed via SOAP. In our case, we will have a method `convertINRToUSD` that will take the amount in INR and return the equivalent USD amount.

java Copy

```
// CurrencyConversionService.java import
javax.jws.WebMethod; import
javax.jws.WebService;
```

```
@WebService
```

```
public interface CurrencyConversionService
{ @WebMethod
    public double convertINRToUSD(double inr);
}
```

Step 2.2: Implement the Web Service

Now, we implement the interface. This is the logic of the service where we will define the conversion rate and logic.

```
// CurrencyConversionServiceImpl.java import
javax.jws.WebService;
```

```
@WebService(endpointInterface = "CurrencyConversionService") public
class CurrencyConversionServiceImpl implements
CurrencyConversionService {
    @Override
    public double convertINRToUSD(double inr) {
        // Hardcoded conversion rate: 1 INR = 0.012 USD return inr *
        0.012;
    }
}
```

Step 2.3: Publish the Web Service

We need to publish the service using the Endpoint class from javax.xml.ws.

```
// CurrencyConversionServer.java import
javax.xml.ws.Endpoint;
```

```
public class CurrencyConversionServer { public static
void main(String[] args) {
    // Publish the service at a specific URL (e.g., http://localhost:8080/convert)
    Endpoint.publish("http://localhost:8080/convert", new
CurrencyConversionServiceImpl()); System.out.println("SOAP Service
is running...");
}
}
```

Now, your SOAP service is running locally on <http://localhost:8080/convert>.

Test the SOAP Service

Step 3.1: Create a SOAP Request XML

You can create a sample SOAP request to test your service. A SOAP request will look like this:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:web="http://webservice.com/">
  <soapenv:Header/>
  <soapenv:Body>
    <web:convertINRToUSD>
      <inr>1000</inr>
    </web:convertINRToUSD>
  </soapenv:Body>
</soapenv:Envelope>
```

To send a SOAP request, you can use tools like **SoapUI** or even a simple **Java client**.

`wsimport -keep http://localhost:8080/convert?wsdl`

`wsimport -keep http://localhost:8080/convert?wsdl`

This will generate the necessary Java classes (including a client class) that you can use to call the service.

Step 4.2: Write a Client to Call the SOAP Web Service

// CurrencyConversionClient.java

```
import javax.xml.namespace.QName; import
javax.xml.ws.Service;
import java.net.URL;
```

```
public class CurrencyConversionClient {
  public static void main(String[] args) throws Exception {
    // The URL of the WSDL file
    URL wsdlURL = new URL("http://localhost:8080/convert?wsdl");

    // Define the QName (namespace and service name)
```



```

    QName serviceName = new QName("http://webservice.com/",
    "CurrencyConversionServiceImplService");

    // Create a service instance
    Service service = Service.create(wsdlURL, serviceName);

    // Get the proxy (stub) of the web service
    CurrencyConversionService port =
    service.getPort(CurrencyConversionService.class);

    // Call the SOAP service method double inr =
    1000;
    double usd = port.convertINRToUSD(inr); System.out.println(inr + "
    INR = " + usd + " USD");
}
}

```

5. Run the Application

1. Run the SOAP Web Service:

- a. Run the CurrencyConversionServer.java class to start the SOAP web service.
- b. The service will be available at <http://localhost:8080/convert>.

Run the SOAP Client:

- c. Run the CurrencyConversionClient.java class to call the service.
- d. It should output the conversion from INR to USD.

Practical 3: Create a Simple REST Service

In this example, we will build a **REST API** using **Spring Boot**, which is a popular framework for creating RESTful services in Java.

Set Up the Spring Boot Application Step 1.1:

Create a Spring Boot Project

You can either use **Spring Initializr** (<https://start.spring.io/>) or manually set up the project using Maven.

For Maven (Add dependencies in pom.xml):

```
<dependencies>

  <!-- Spring Web dependency for RESTful web services -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-web</artifactId>

  </dependency>

  <!-- Spring Boot dependency -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter</artifactId>

  </dependency>

</dependencies>
```

Step 1.2: Create the Main Application Class

Create a main class to launch the Spring Boot application.

```
// CurrencyConversionApplication.java package
com.example.currencyconversion;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class CurrencyConversionApplication { public static
void main(String[] args) {
    SpringApplication.run(CurrencyConversionApplication.class, args);
```

```
}
```

```
}
```

This class is responsible for launching the Spring Boot application.

Define the REST Controller

Now, we will define the **REST API** to convert INR to USD. This controller will have a method that accepts an INR amount and returns the equivalent USD amount.

Step 2.1: Create the Controller

```
// CurrencyConversionController.java package  
com.example.currencyconversion;
```

```
import org.springframework.web.bind.annotation.GetMapping; import  
org.springframework.web.bind.annotation.RequestParam; import  
org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class CurrencyConversionController {  
    @GetMapping("/convert")
```

```
    public double convertINRToUSD(@RequestParam double inr) {
```

```
        // Hardcoded conversion rate: 1 INR = 0.012 USD double  
        conversionRate = 0.012;  
        return inr * conversionRate;
```

```
    }
```

```
}
```

Explanation:

- `@RestController`: This annotation marks the class as a RESTful controller.
- `@GetMapping("/convert")`: This annotation tells Spring to map HTTP GET requests to this method.
- `@RequestParam double inr`: This is the query parameter that we will pass in the URL (for example: `/convert?inr=1000`).
- The method takes the amount in INR, converts it to USD using a hardcoded conversion rate, and returns the result.

3. Run the Application

Step 3.1: Run the Spring Boot Application

- Once the application is set up, you can run it using your IDE (such as IntelliJ IDEA or Eclipse) or by running the following command in the terminal if you are using Maven:

```
mvn spring-boot:run
```

Alternatively, you can run the `CurrencyConversionApplication.java` class from your IDE by right-clicking and selecting **Run**.

Test the REST Service

Now that the application is running, you can test it using:

1. **Browser:** Open a browser and navigate to `http://localhost:8080/convert?inr=1000`. You should see a response with the converted USD value.

Example:

12.0

- (Since $1000 \text{ INR} = 1000 * 0.012 \text{ USD}$)
- **Postman or cURL:** You can also use **Postman** or **cURL** to test the service. Example cURL command:

```
curl http://localhost:8080/convert?inr=1000
```

Final Project Structure

The final structure of your project should look like this: pgsq
curl "http://localhost:8080/convert?inr=1000"

Final Project Structure

The final structure of your project should look like this: pgsq
currency-conversion

```
|
|
|— src
|  |— main
|  |  |— java
|  |  |  |— com
```

```
| | └─ example
| | └─ currencyconversion
| | └─ CurrencyConversionApplication.java
| | └─ CurrencyConversionController.java
| └─ resources
|   └─ application.properties
└─ pom.xml
```

Conclusion

You've now created a simple RESTful web service in Java using Spring Boot. Here's a quick summary of the steps:

1. **Set up the Spring Boot project** with the necessary dependencies.
2. **Define the REST controller** that implements the conversion logic.
3. **Run the application** and test it via the browser, Postman, or cURL.
4. **Optional enhancements:** You could add error handling, validation, or real-time exchange rates by integrating with external APIs.

Practical 4: Develop an application to Consume Google's Search/Google Maps RESTful Web Service

1. Set Up Google Cloud API Access

Before you can consume Google's APIs, you need to:

Create a Google Cloud project:

- a. Go to Google Cloud Console.
- b. Create a new project (or use an existing one).

Enable the API:

- c. Go to the **API G Services** section.
- d. Click on **Enable APIs and Services**.
- e. Search for **Geocoding API** (for Maps-related services) or **Custom Search API** (for search-related services), and enable it.

Create an API key:

- f. In **API G Services > Credentials**, click **Create credentials**.
- g. Choose **API key**. You will use this key to authenticate requests to the Google API.
- h. Note down the API key generated.

Set Up the Project Environment

For this example, let's use **Java** and **Spring Boot** to consume the Google Maps Geocoding API. You can use the following dependencies in your **Maven** project.

Maven Dependencies (add to your pom.xml):

```
<dependencies>

<!-- Spring Boot Starter Web for RESTful services -->

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-web</artifactId>

</dependency>

<!-- Jackson for JSON parsing -->

<dependency>

    <groupId>com.fasterxml.jackson.core</groupId>

    <artifactId>jackson-databind</artifactId>

</dependency>

<!-- Spring Boot Starter -->

<dependency>
```

```
<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter</artifactId>

</dependency>

</dependencies>
```

Alternatively, if you prefer to use **Gradle**, you can add: gradle dependencies {

```
    implementation 'org.springframework.boot:spring-boot-starter-web' implementation
    'com.fasterxml.jackson.core:jackson-databind'
}
```

3. Create the REST Client to Consume the Google Maps API

Now we will create the Java application that will make HTTP requests to the Google Geocoding API and process the response.

Step 3.1: Create a Spring Boot Application

```
// GoogleMapsApplication.java package
com.example.googlemaps;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class GoogleMapsApplication { public static
    void main(String[] args) {
        SpringApplication.run(GoogleMapsApplication.class, args);
    }
}
```

Step 3.2: Create a Controller to Handle the API Requests

Create a controller to interact with the Google Maps API and process incoming HTTP requests. This controller will accept an address and fetch the corresponding coordinates.

```
// GoogleMapsController.java package
com.example.googlemaps;
```

```
import org.springframework.beans.factory.annotation.Value; import
org.springframework.web.bind.annotation.GetMapping; import
```

```
org.springframework.web.bind.annotation.RequestParam; import
org.springframework.web.bind.annotation.RestController; import
org.springframework.web.client.RestTemplate;
```

```
import java.util.Map;
```

```
@RestController
```

```
public class GoogleMapsController {
```

```
    @Value("${google.api.key}") private
    String googleApiKey;
```

```
    @GetMapping("/geocode")
```

```
    public String geocodeAddress(@RequestParam String address) {
```

```
        // URL to call Google Maps Geocoding API
```

```
        String url = "https://maps.googleapis.com/maps/api/geocode/json?address=" + address +
        "Ckey=" + googleApiKey;
```

```
        // Create a RestTemplate to make the request RestTemplate
        restTemplate = new RestTemplate();
```

```
        // Send GET request to Google Maps Geocoding API and get the response String
        response = restTemplate.getForObject(url, String.class);
```

```
        // Return the raw JSON response for now return
        response;
    }
```

```
}
```

Explanation:

- `@Value("${google.api.key}")`: This annotation loads your Google API key from the application.properties file.
- `@GetMapping("/geocode")`: This method will be triggered when you send a GET request to /geocode with an address parameter (e.g.,

/geocode?address=New+York).

- We use the **RestTemplate** to make an HTTP GET request to the Google Maps API and return the JSON response.

4. Add Your Google API Key

- In the `src/main/resources/application.properties` file, add your API key like this:

```
google.api.key=YOUR_GOOGLE_API_KEY
```

Make sure to replace `YOUR_GOOGLE_API_KEY` with the actual API key you generated earlier.

Test the Service

- **Run the Spring Boot application:**
 - In your terminal, navigate to the project folder and run:

```
mvn spring-boot:run
```

- run the `GoogleMapsApplication.java` class directly from your IDE.
- **Call the Service:**
 - Open your browser or use **Postman** or **cURL** to make a GET request to your local server:

Example using cURL:

```
curl http://localhost:8080/geocode?address=New+York
```

Sample JSON Response (simplified version): json

```
{
  "results": [
    {
      "geometry": {
        "location": {
          "lat": 40.712776,
          "lng": -74.005974
        }
      }
    }
  ]
}
```

```
}  
  
    }  
  ],  
  "status": "OK"  
}
```

Parse and Display Latitude and Longitude

To make the output more user-friendly, let's parse the JSON response and extract the latitude and longitude.

Step 6.1: Create a POJO to Map the JSON Response

You can use **Jackson** to parse the JSON into Java objects. First, create a POJO class that represents the response structure.

```
// GeocodingResponse.java package  
com.example.googlemaps;
```

```
public class GeocodingResponse { private  
    Result[] results;
```

```
  
    public Result[] getResults() { return  
        results;  
    }
```

```
  
    public void setResults(Result[] results)  
        { this.results = results;  
  
    }
```

```
  
    public static class Result { private  
        Geometry geometry;
```

```
  
        public Geometry getGeometry() { return  
            geometry;  
        }
```

```
  
        public void setGeometry(Geometry geometry) { this.geometry  
            = geometry;  
        }  
    }
```

```
  
    public static class Geometry { private  
        Location location;
```

```
  
        public Location getLocation() { return  
            location;  
        }
```

```
  
        public void setLocation(Location location) { this.location =  
            location;  
        }  
    }
```

```

public static class Location { private
    double lat;
    private double lng;

    public double getLat() { return lat;
    }

    public void setLat(double lat) { this.lat =
        lat;
    }

    public double getLng() { return lng;
    }

    public void setLng(double lng) { this.lng =
        lng;
    }
}

```

Step 6.2: Modify the Controller to Parse JSON Response Modify the controller to return the parsed latitude and longitude: java
 @GetMapping("/geocode")

```

public String geocodeAddress(@RequestParam String address) {

    String url = "https://maps.googleapis.com/maps/api/geocode/json?address=" + address
    + "Ckey=" + googleApiKey;

    RestTemplate restTemplate = new RestTemplate();

    // Parse JSON into GeocodingResponse object GeocodingResponse
    response = restTemplate.getForObject(url,
    GeocodingResponse.class);

    // Get latitude and longitude

    if (response != null CC response.getResults().length > 0) {

```

```

double lat = response.getResults()[0].getGeometry().getLocation().getLat(); double
lng = response.getResults()[0].getGeometry().getLocation().getLng(); return
"Latitude: " + lat + ", Longitude: " + lng;
} else {

    return "No results found for the given address.";

}
}

```

Conclusion

In this example, we created a simple **Spring Boot application** that consumes the **Google Maps Geocoding API** to convert an address into latitude and longitude. We followed these steps:

1. **Set up a Google Cloud project** and enable the required APIs.
2. **Create a Spring Boot project** and build a REST controller to call the Google Maps API.
3. **Parse the JSON response** and return the latitude and longitude.

Practical: Installation and configuration of Virtualization using KVM

Objectives: From this experiment, the student will be able to,

- Understand the concepts of virtualization.
- Understand KVM architecture and its configuration.

Outcomes: The learner will be able,

- To analyze user models and develop user centric interfaces
- To analyze the local and global impact of computing on individuals, organizations, and society.
- To engage in life-long learning development and higher studies.
- To understand, identify, analyze and design the problem, implement and validate the solution including both hardware and software.

Hardware / Software Required: Ubuntu operating system, open source software KVM, Internet.

Theory:

Virtualization is software that separates physical infrastructures to create various dedicated resources. It is the fundamental technology that powers cloud computing.

The technology behind virtualization is known as a virtual machine monitor (VMM) or virtual manager, which separates compute environments from the actual physical infrastructure.

Virtualization makes servers, workstations, storage and other systems independent of the physical hardware layer. This is done by installing a Hypervisor on top of the hardware layer, where the systems are then installed.

There are three areas of IT where virtualization is making headroads, network virtualization, storage virtualization and server virtualization:

Network virtualization is a method of combining the available resources in a network by splitting up the available bandwidth into channels, each of which is independent from the others, and each of which can be assigned (or reassigned) to a particular server or device in real time. The idea is that virtualization disguises the true complexity of the network by separating it into manageable parts, much like your partitioned hard drive makes it easier to manage your files. Storage virtualization is the pooling of physical storage from multiple network storage devices into what appears to be a single storage device that is managed from a central console. Storage virtualization is commonly used in storage area networks (SANs).

Server virtualization is the masking of server resources (including the number and identity of individual physical servers, processors, and operating systems) from server users. The intention is to spare the user from having to understand and manage complicated details of server resources while increasing resource sharing and utilization and maintaining the capacity to expand later. Virtualization can be viewed as part of an overall trend in enterprise IT that includes autonomic computing, a scenario in which the IT environment will be able to manage itself based on perceived activity, and utility computing, in which computer processing power is seen as a utility that clients can pay for only as needed. The usual goal of virtualization is to centralize administrative tasks while improving scalability and work loads the true complexity of the network by separating it into manageable parts, much like your partitioned hard drive makes it easier to manage your files.

Storage virtualization is the pooling of physical storage from multiple network storage devices into what appears to be a single storage device that is managed from a central console. Storage virtualization is commonly used in storage area networks (SANs).

Server virtualization is the masking of server resources (including the number and identity of individual physical servers, processors, and operating systems) from server users. The intention is to spare the user from having to understand and manage complicated details of server resources while increasing resource sharing and utilization and maintaining the capacity to expand later.

Virtualization can be viewed as part of an overall trend in enterprise IT that includes autonomic computing, a scenario in which the IT environment will be able to manage itself based on perceived activity, and utility computing, in which computer processing power is seen as a utility that clients can pay for only as needed. The usual goal of virtualization is to centralize administrative tasks while improving scalability and work loads.

Procedure:

Installation Steps :

1.

`sudo grep -c "svm\|vmx" /proc/cpuinfo` 2.

`sudo apt-get install qemu-kvm libvirt-bin bridge-utils virt-manager` 3.

`sudo adduser rait` 4.

`sudo adduser rait libvirtd`

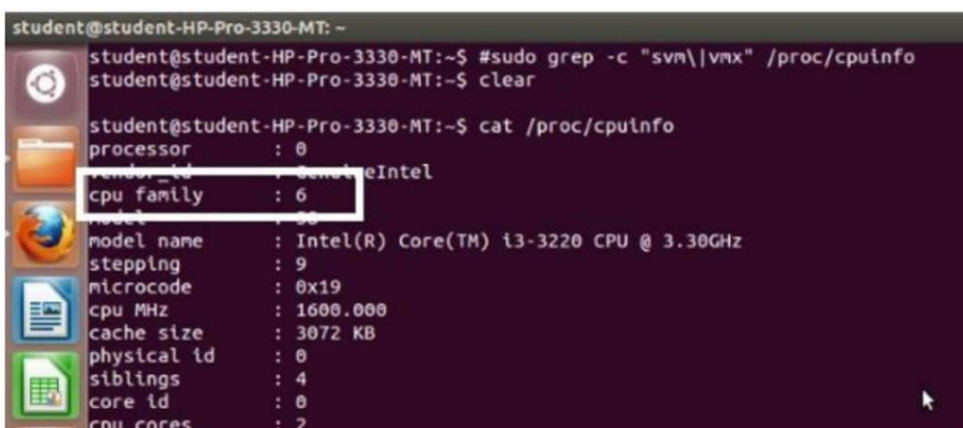
5. `virsh -c qemu:///system list`

(Run following command after logging back in as rait and you should see an empty list of virtual machines. This indicates that everything is working correctly.)

6. `virt-manager`

(Open Virtual Machine Manager application and Create Virtual Machine)

Step 1 : `#sudo grep -c "svm\|vmx" /proc/cpuinfo`



```
student@student-HP-Pro-3330-MT: ~
student@student-HP-Pro-3330-MT:~$ #sudo grep -c "svm\|vmx" /proc/cpuinfo
student@student-HP-Pro-3330-MT:~$ clear

student@student-HP-Pro-3330-MT:~$ cat /proc/cpuinfo
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 58
model name     : Intel(R) Core(TM) i3-3220 CPU @ 3.30GHz
stepping       : 9
microcode      : 0x19
cpu MHz        : 1600.000
cache size     : 3072 KB
physical id    : 0
siblings       : 4
core id        : 0
cpu cores      : 2
```

Step 2 : #sudo apt-get install qemu-kvm libvirt-bin bridge-utils virt-manager

```
fpu : yes
fpu_exception : yes
cpuid level : 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36
scp lm constant_tsc arch_perfmon pebs bts xtopology nonstop_tsc aperfmperf pni pclmulqdq d
pcid sse4_1 sse4_2 popcnt tsc_deadline_timer xsave avx f16c lahfm arat epb xsaveopt pln
ase snep erns
bogomips : 6584.72
clflush size : 64
cache_alignment : 64
address sizes : 36 bits physical, 48 bits virtual
power management:

student@student-HP-Pro-3330-MT:~$ #sudo grep -c "svm|vmx" /proc/cpuinfo
student@student-HP-Pro-3330-MT:~$ sudo grep -c "svm|vmx" /proc/cpuinfo
[sudo] password for student:
4
student@student-HP-Pro-3330-MT:~$ sudo apt-get update
Ign http://repo.mongodb.org precise/mongodb-org/3.2 InRelease
Ign http://extras.ubuntu.com precise InRelease
Hit http://repo.mongodb.org precise/mongodb-org/3.2 Release.gpg
Hit http://repo.mongodb.org precise/mongodb-org/3.2 Release
Ign http://in.archive.ubuntu.com precise InRelease
Get:1 http://in.archive.ubuntu.com precise-updates InRelease [55.7 kB]
Hit http://extras.ubuntu.com precise Release.gpg
Get:2 http://security.ubuntu.com precise-security InRelease [55.7 kB]
Hit http://repo.mongodb.org precise/mongodb-org/3.2/multiverse i386 Packages
Hit http://extras.ubuntu.com precise Release
```

Step 3 : #sudo adduser rait

After running this command, log out and log back in as rait

```
Hit http://in.archive.ubuntu.com precise/restricted TranslationIndex
Hit http://in.archive.ubuntu.com precise/universe TranslationIndex
Hit http://in.archive.ubuntu.com precise-backports/main Translation-en
Hit http://in.archive.ubuntu.com precise-backports/multiverse Translation-en
Hit http://in.archive.ubuntu.com precise-backports/restricted Translation-en
Hit http://in.archive.ubuntu.com precise-backports/universe Translation-en
Hit http://in.archive.ubuntu.com precise/main Translation-en
Hit http://in.archive.ubuntu.com precise/multiverse Translation-en
Hit http://in.archive.ubuntu.com precise/restricted Translation-en
Hit http://in.archive.ubuntu.com precise/universe Translation-en
Fetched 4,065 kB in 39s (103 kB/s)
Reading package lists... Done
student@student-HP-Pro-3330-MT:~$ sudo apt-get install qemu-kvm libvirt-bin bridge-utils virt-manager
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following extra packages will be installed:
  cgroup-lite cpu-checker ebttables gawk kvm-tpxe libatol libapparmor1 libbonoboui2-0 libbonoboui2-common libglade2-0 l
  libgnomecanvas2-common libgnomeui-0 libgnomeui-common libgtk-vnc-1.0-0 libgvnc-1.0-0 libnuma1 librados2 librbdi librs
  libvte-common libvte9 libxext3 libxext3 libxml2-utils msr-tools python-glade2 python-gnome2 python-gtk-vnc python-libvt
  python-urgrabber python-vte qemu-common qemu-utils seabios vgabios virtinst
Suggested packages:
  libvirt-clients libvirt-daemon-kvm libvirt-daemon-system libvirt-docs qemu-kvm-bin seabios
The following NEW packages will be installed:
  bridge-utils libvirt-bin qemu-kvm virt-manager
0 upgraded, 4 newly installed, 0 to remove and 0 not upgraded.
Need to get 10.5 MB of archives.
After this operation, 40.5 MB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com precise/main bridge-utils amd64 1.6.2-1 [28.5 kB]
Get:2 http://in.archive.ubuntu.com precise/main libvirt-bin amd64 1.0.6-1 [10.5 MB]
Get:3 http://in.archive.ubuntu.com precise/main qemu-kvm amd64 1:1.2.2-1 [10.5 MB]
Get:4 http://in.archive.ubuntu.com precise/main virt-manager amd64 1:1.2.2-1 [10.5 MB]
Fetched 10.5 MB in 39s (268 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Setting up bridge-utils (1.6.2-1) ...
Setting up libvirt-bin (1.0.6-1) ...
Setting up qemu-kvm (1:1.2.2-1) ...
Setting up virt-manager (1:1.2.2-1) ...
```


Step 4 : #sudo adduser rait libvirt

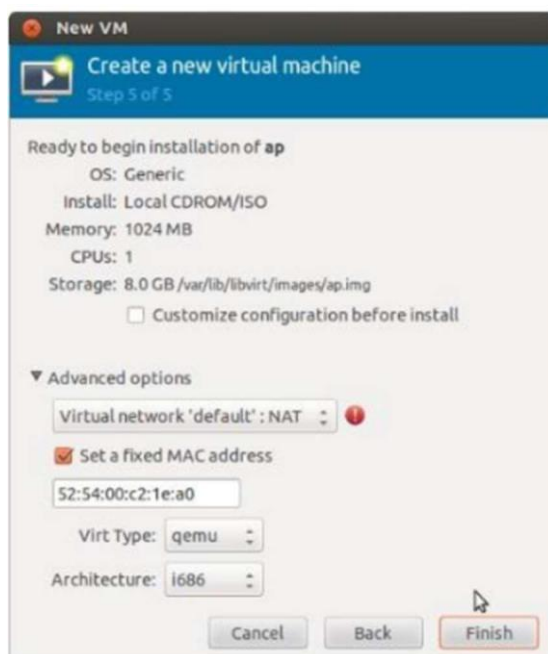
After running this command, log out and log back in as rait

```
Setting up python-vte (1:0.28.2-3ubuntu2) ...  
Setting up virtinst (0.600.1-1ubuntu3.3) ...  
Setting up virt-manager (0.9.1-1ubuntu5.1) ...  
Processing triggers for libc-bin ...  
ldconfig deferred processing now taking place  
student@student-HP-Pro-3330-MT:~$ virt-manager  
student@student-HP-Pro-3330-MT:~$
```

Step 5 : Open Virtual Machine Manager application and Create Virtual Machine
#virt-manager as shown below



Step 6 : Create a new virtual machine as shown below

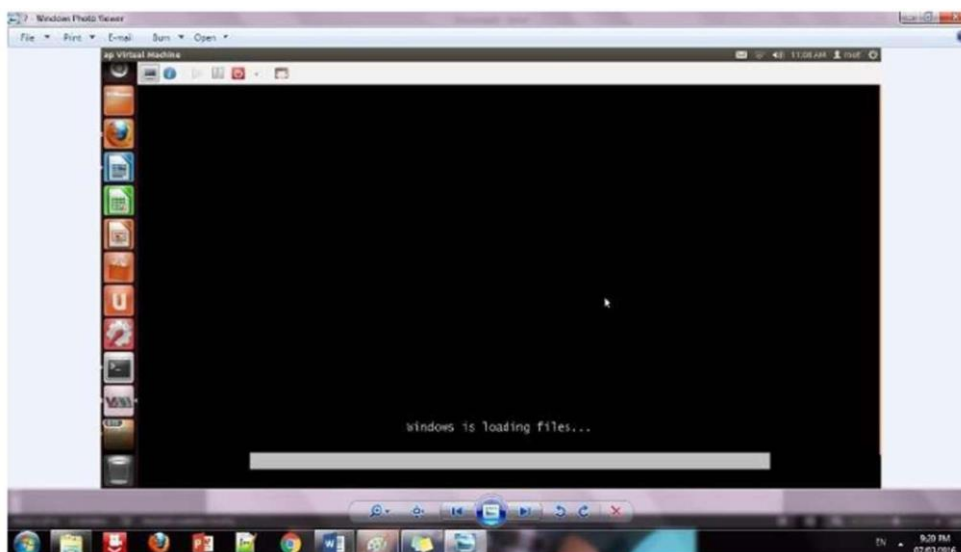


Step 7 : Install windows operating system on virtual machine

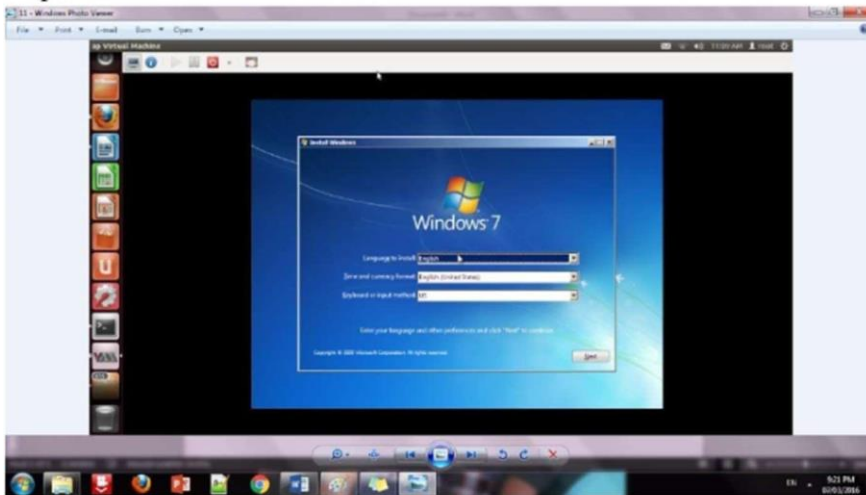


Step 7 : Install windows operating system on virtual machine

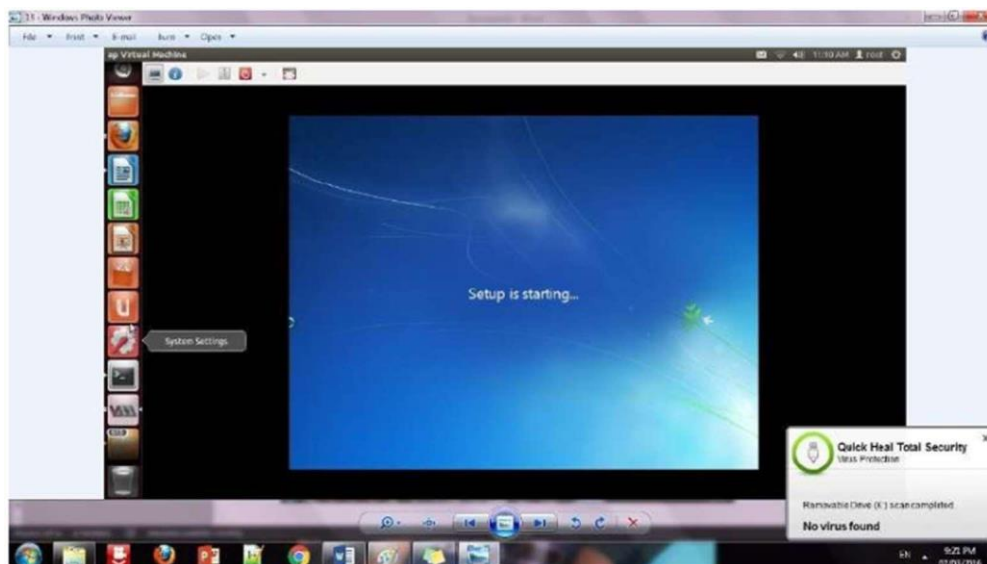
Step 8: Installation of windows on virtual machine



Step 9: Installation of windows 7 on virtual machine



Step 10: Initialization of windows on virtual machine



6. Implementation of open stack with user C private network creation

Prerequisites:

1. **OpenStack Installation:** Ensure that OpenStack is already installed. This can be done through distributions like DevStack for development environments or RDO for production deployments.
2. **Admin Access:** You need administrative privileges to create users and networks.
3. **Networking Plugin:** OpenStack typically uses Neutron for networking, and this can be configured with various backends such as Linux Bridge or Open vSwitch (OVS).

Steps for User G Private Network Creation:

1. Set Up OpenStack Networking (Neutron)

OpenStack uses **Neutron** for managing networks. First, ensure that Neutron is installed and correctly configured.

- In a production environment, you may need to configure the Neutron service to use the backend (e.g., LinuxBridge, Open vSwitch, or other plugins).

2. Creating a Private Network

Once Neutron is up and running, you can create a private network in OpenStack.

Steps:

1. **Create a Network:** This network will be the private network for users.

```
openstack network create private-net
```

This creates a network named private-net.

2. **Create a Subnet:** Assign a subnet to the network. You should specify the CIDR block for the private network.

```
openstack subnet create --network private-net --subnet-range 192.168.1.0/24  
private-subnet
```

This creates a subnet named private-subnet on private-net with a CIDR range of 192.168.1.0/24.

3. **Create a Router:** To allow communication between your private network and other networks (like public networks), you need to create a router.

```
openstack router create router1
```

4. **Attach Router to Private Network:** Attach the router to the private network. `openstack router add subnet router1 private-subnet`

5. **Create a Floating IP Pool (Optional):** If you want your users to access the private network externally, you need to assign floating IPs. If the environment supports it (and there is a public network configured), create a floating IP pool and associate it with the router:

```
openstack router set --external-gateway public-net router1
```

3. Creating a User

You can create a new user in OpenStack and assign it appropriate roles.

Steps:

1. **Create a User:**

```
openstack user create --domain default --password <password> newuser
```

2. **Assign Role to User:** After creating the user, assign them a role in the appropriate project. For example, assigning the Member role in the admin project.

```
openstack role add --project admin --user newuser --role member
```

3. **Create a Project (if required):** If needed, create a new project where the user can launch instances.

```
openstack project create new_project
```

4. **Assign User to Project:** You may need to assign the user to a project and set permissions. : `openstack role add --project new_project --user newuser --role member`

Launch an Instance: When launching an instance, specify that it should be connected to the private network: `openstack server create --flavor m1.small -- image <image-id> --key-name my_key --network private-net my_instance`

Check the Instance: Ensure the instance is running and connected to the private network : `openstack server list`

5. User Access to Networks

- The user will now have access to the private-net network and can deploy instances into it.
- If you created floating IPs, users can associate floating IPs to their instances to allow external access.

6. Verify Private Network Access

You can verify that the instances in the private network can communicate with each other and, if configured, the outside world.

1. **Accessing the Instance:** To access the instance in the private network, either use an internal network connection or associate a floating IP with the instance and SSH into it.
2. **Test Connectivity:** SSH into one instance and try to ping other instances or external networks

7 To create a simple workflow application using the AWS Step Functions

Prerequisites

Before starting, you will need:

1. **AWS Account:** Access to an AWS account.
2. **AWS CLI:** AWS Command Line Interface (CLI) installed and configured.
3. **IAM Roles and Permissions:** Ensure you have appropriate permissions to create and manage Step Functions, Lambda functions, and other AWS services.
4. **AWS Lambda Function:** Basic knowledge of AWS Lambda, as workflows often integrate Lambda functions to handle tasks.

Steps for Developing a Simple Workflow Using AWS Step Functions

In this example, we'll create a simple workflow that performs the following:

1. Invoke a Lambda function to process some input.
2. Pass the output of the Lambda function to another Lambda function that performs another task.

Step 1: Create Two AWS Lambda Functions

First, create two Lambda functions that will be used in the workflow. These functions will simulate two simple steps in your workflow.

Lambda Function 1 (Function A)

1. Go to the AWS Lambda Console.
2. Create a new function: Name it FunctionA, and select a runtime like Python, Node.js, or any runtime of your choice.
3. Function Code: Example Python function: python

```
def lambda_handler(event, context): message
    = "Hello from Function A!" return {

        'statusCode': 200, 'body':
        message
    }
```

1. Deploy the function.

Lambda Function 2 (Function B)

1. Go to the AWS Lambda Console.
2. Create a new function: Name it FunctionB, and choose the runtime.
3. Function Code: Example Python function: def lambda_handler(event, context):

```
    message = event.get("body", "") + " Now in Function B!" return {
        'statusCode': 200, 'body':
        message
    }
```

Deploy the function.

Step 2: Create the Step Functions State Machine

Now, let's define a simple workflow using AWS Step Functions. This workflow will invoke FunctionA first and then pass the output to FunctionB.

1. Go to the AWS Step Functions Console.
2. Click on Create state machine.
3. Choose Author with code snippets to define your workflow using Amazon States Language (ASL).
4. Write the workflow definition using the following JSON:

```
{
  "Comment": "A simple workflow with two Lambda functions",

  "StartAt": "Invoke Function A",
  "States": {
    "Invoke Function A": { "Type":
      "Task",
      "Resource": "arn:aws:lambda:REGION:ACCOUNT_ID:function:FunctionA", "Next":
        "Invoke Function B"
    },

    "Invoke Function B": { "Type":
      "Task",
      "Resource": "arn:aws:lambda:REGION:ACCOUNT_ID:function:FunctionB", "End": true
    }
  }
}
```

- Replace REGION and ACCOUNT_ID with your actual AWS region and account ID.
- Name your State Machine (e.g., SimpleWorkflow).
- Click Create State Machine.

Step 3: Execute the Workflow

To start the workflow, you can use the AWS Step Functions Console, AWS CLI, or AWS SDKs to trigger the execution.

Using the AWS Console:

1. Go to the Step Functions Console.
2. Click on your state machine (e.g., SimpleWorkflow).
3. Click Start Execution.
4. In the input JSON, you can provide a simple object (for example):

```
{
  "body": "Initial message"
```


}

1. Click Start Execution to begin. Using AWS CLI:

You can also trigger the workflow execution using the AWS CLI by running the following command:

```
aws stepfunctions start-execution --state-machine-arn
arn:aws:states:REGION:ACCOUNT_ID:stateMachine:SimpleWorkflow --input '{"body":
"Initial message"}'
```

Step 4: Monitor the Workflow

You can track the execution progress and see the results of each step:

1. Go to the Step Functions Console.
2. Find the execution history for your workflow.

8. Implementing a FOSS (Free and Open-Source Software) cloud functionality as a VSI (Virtual Server Infrastructure) Platform as a Service

Key Components of FOSS VSI Platform as a Service:

1. Virtualization Layer: This layer abstracts the physical hardware and enables the creation, management, and provisioning of virtual machines (VMs).
 - a. KVM (Kernel-based Virtual Machine), Xen, or VirtualBox can be used as the hypervisor.
2. Cloud Orchestration Layer: This is responsible for managing the lifecycle of virtual instances, from provisioning to scaling.
 - a. OpenStack, CloudStack, or oVirt can provide this functionality.
3. Networking Layer: Virtual networks need to be created to provide communication between VMs, internal networks, and external networks.
 - a. Open vSwitch (OVS) or LinuxBridge are commonly used in cloud networks.
4. Storage Layer: You will need to manage persistent storage volumes for your virtual machines.
 - a. Ceph, GlusterFS, or NFS can serve as storage backends.
5. User Interface: A self-service portal is needed for users to request, manage, and monitor virtual machines.
 - a. Horizon (for OpenStack), CloudStack UI, or a custom-built UI can be utilized.
6. APIs: A RESTful API to enable programmatic interaction with the platform.
 - a. OpenStack APIs or CloudStack API can be used for this purpose.

7. Authentication and Authorization: Secure access management and authorization for users to provision resources.
 - a. LDAP, OAuth, or Keystone (for OpenStack) can manage user authentication.

Steps to Implement FOSS VSI Platform as a Service Step 1: Setting Up the Hypervisor Layer

1. Install KVM (Kernel-based Virtual Machine):
 - a. KVM is an open-source hypervisor for Linux that allows you to run virtual machines.
 - b. Install KVM on a Linux server by running: `sudo apt-get update`
`sudo apt-get install qemu-kvm libvirt-bin virtinst bridge-utils`
- ☐ Enable the virtualization in the BIOS/UEFI and ensure that `kvm` and `qemu` modules are loaded.
- ☐ Create Virtual Machines:
- You can use tools like `virt-manager`, `virsh`, or `cloud-init` to configure and launch virtual machines. Alternatively, you can automate VM creation using orchestration tools like OpenStack.

Step 2: Setting Up Cloud Orchestration (OpenStack or CloudStack)

OpenStack and CloudStack are the most popular FOSS tools for cloud orchestration. Below, I will describe the setup for OpenStack, but similar steps apply to CloudStack.

1. Install OpenStack:
 - a. Use DevStack (for development environments) or Packstack (for a production-like environment).
 - b. For DevStack, simply clone the repository and run the installation script: `bash`
`git clone https://git.openstack.org/openstack-dev/devstack cd devstack`
`./stack.sh`
- ☐ This will install and configure all OpenStack components, including Keystone (identity), Nova (compute), Neutron (networking), and Glance (image management).
- ☐ Configure OpenStack Compute (Nova):

- In OpenStack, the Nova service is responsible for provisioning and managing virtual servers.
- After installation, you can use nova commands to manage your instances.
- Example command to launch a new instance

```
openstack server create --flavor m1.small --image <image-id> --network
```

Step 3: Setting Up Networking (Neutron)

1. Configure Neutron:

- Neutron is OpenStack's networking service that allows you to create private networks, manage routers, and assign floating IPs for your VMs.
- Example commands for creating a private network and subnet: openstack network create private-net

```
openstack subnet create --network private-net --subnet-range 10.168.1.0/24
private-subnet
```

- □ After creating the network, associate it with your compute instances.
- Router Setup:
- Attach a router to connect the private network to an external network (i.e., the public Internet)

```
openstack router create router1
```

```
openstack router add subnet router1 private-subnet openstack
router set --external-gateway public-net router1
```

Step 4: Storage Management (Cinder, Ceph, or GlusterFS)

For persistent storage, OpenStack provides Cinder for block storage, but you can also integrate distributed storage systems like Ceph or GlusterFS.

1. Cinder Setup:

- a. Install the Cinder service to provide block storage:

```
sudo apt-get install cinder-api cinder-scheduler cinder-volume
```

- □ Configure Cinder to use backend storage (e.g., Ceph, LVM, NFS).
- Ceph Setup (if using Ceph for distributed storage):
- Ceph provides a scalable and reliable storage system for OpenStack.

Step 5: User Portal and API Access

1. Self-Service Portal:

- If you are using OpenStack, the Horizon dashboard is a built-in web interface for users to launch instances, manage networks, and handle storage.
- Example of accessing the Horizon UI: <http://<your-openstack-server>/dashboard>
- - Users can log in, provision VMs, and manage resources from the UI.
 - API Access:
- You can expose REST APIs for programmatic access to the platform. OpenStack provides extensive APIs for all services (e.g., compute, networking, storage).